

Observabilidad con OpenTelemetry: correlación de logs, métricas y trazas en AlimentaCR

Kevin Salazar Bravo, Kimberley Gómez Quesada

Escuela de Informática, Universidad Nacional de Costa Rica

kimberley.gomez.quesada@est.una.ac.cr · Kevin.salazar.bravo@est.una.ac.cr

EIF509 Desarrollo de Aplicaciones Basadas en Web · II Ciclo 2026 · Prof. Elberth Adrián Garro Sánchez

Resumen—En una aplicación web con varias capas y bases de datos, encontrar la causa de un error o de una lentitud es difícil, porque una sola solicitud recorre muchas partes del sistema y los registros, las métricas y las trazas suelen revisarse por separado. Este artículo analiza cómo OpenTelemetry, un marco de código abierto para generar y exportar datos de telemetría, permite relacionar esas tres señales mediante la propagación de contexto, y compara tres estrategias de instrumentación: automática, manual e híbrida. La comparación usa cuatro criterios técnicos (nivel de detalle, complejidad y mantenimiento, rendimiento, y seguridad y privacidad) y se apoya en documentación técnica oficial y en un estudio reciente sobre el costo de rastrear. Los hallazgos indican que la automática es la más fácil de adoptar, pero no muestra el código propio; que la manual da más detalle a cambio de más esfuerzo; y que toda estrategia que agrega puntos de medición aumenta el costo en rendimiento. Se aplica el análisis a AlimentaCR, un backend para gestionar donaciones de alimentos, desarrollado con Django REST Framework, PostgreSQL y MongoDB, y se propone una estrategia híbrida con un piloto en la publicación de donaciones. La propuesta no se ha implementado ni medido, por lo que sus beneficios requieren validación.

Palabras clave—observabilidad, OpenTelemetry, trazas distribuidas, instrumentación, Django.

I. INTRODUCCIÓN

Cuando una aplicación web crece, encontrar la causa de un error se vuelve bastante difícil, ya que una sola solicitud suele pasar por muchas partes del código, y los mensajes de error no siempre indican exactamente dónde está el problema. En AlimentaCR, por ejemplo, publicar una donación recorre la vista, el servicio, PostgreSQL y MongoDB, y si ocurre un error inesperado, el usuario recibe un mensaje genérico que no indica dónde falló. Los registros (logs) por sí solos suelen carecer de información contextual para reconstruir la ejecución, y resultan mucho más útiles cuando se relacionan con una traza y un span [1]. Sin esa relación, hay que revisar a mano registros, métricas y trazas por separado, lo que consume tiempo.

OpenTelemetry (OTel) es un marco de código abierto e independiente del proveedor que permite instrumentar un

sistema y exportar datos de telemetría como trazas, métricas y registros (logs) [2]. Estos datos ayudan a que un sistema sea observable, es decir, a entenderlo desde afuera y solucionar problemas imprevistos [1]. La propagación de contexto permite relacionar entre sí las señales que genera el sistema: cada solicitud lleva identificadores de traza y de span que se transmiten entre componentes [3]. Para producir las señales, OTel ofrece dos vías. La instrumentación basada en código (manual) usa la API y el SDK y permite obtener información más detallada. La de sin código (automática) es útil para empezar o cuando no se puede modificar la aplicación, y entrega telemetría dependiendo de las bibliotecas que se usan o del entorno donde se ejecuta [4].

Este artículo analiza cómo OTel relaciona registros, métricas y trazas, y compara la instrumentación automática, manual e híbrida con criterios técnicos muy claros. El análisis será aplicado

a AlimentaCR, un backend desarrollado con Django REST Framework, PostgreSQL y MongoDB. La pregunta que guía el trabajo es: ¿qué tipo de instrumentación con OpenTelemetry conviene usar en AlimentaCR para diagnosticar errores y problemas de rendimiento sin complicar el proyecto?

El documento se organiza así: la sección II presenta el marco conceptual, la III compara las estrategias de instrumentación, la IV las aplica a AlimentaCR y la V presenta las conclusiones.

II. MARCO CONCEPTUAL

A. Trazas y spans

Un span representa una unidad de trabajo u operación y es la pieza con la que se construyen las trazas. Cada span guarda su nombre, el ID de su span padre (vacío en el span raíz), las marcas de tiempo tanto de inicio como del fin, su contexto, atributos, eventos, enlaces (relacionan un span con otros spans) y un estado. Los atributos son metadatos que dan información extra sobre la operación que se rastrea, y el

estado puede ser *Unset* (la operación terminó sin error), *Error* (ocurrió un error, como un error 500 del servidor) u *Ok* (el desarrollador lo marcó explícitamente como exitoso). Los spans pueden anidarse, de modo que un span hijo representa una suboperación de su padre. Una traza es el conjunto de spans que registra el recorrido de una solicitud realizada en la aplicación: todos comparten el mismo `trace_id`, y cada uno tiene su propio `span_id` por así decirlo es su id único. El span raíz es el único que no tiene padre [5].

B. Registros y métricas

Un registro (*log*) es un registro de texto con marca de tiempo, preferiblemente estructurado, que puede incluir metadatos opcionales [6]. OTel ofrece una API y un SDK para producirlos, y se integra con las bibliotecas de logging existentes mediante adaptadores [6]. Los eventos son un tipo especial de registro: todo evento es un registro, pero no todo registro es un evento [6]. Cada registro puede llevar el identificador de la traza y del span, y al activar la instrumentación automática o el SDK, OTel los correlaciona con la solicitud en curso [6]. Esto permite relacionar un registro con la solicitud y con el paso exacto donde se generó, y desde ahí consultar la traza completa. En AlimentaCR, por ejemplo, si al publicar una donación ocurre un error inesperado, hoy el error queda en el log del manejador global sin ningún dato que lo relacione con lo que pasó antes. Si ese registro incluyera el `trace_id`, se podría abrir la traza de esa solicitud y ver si el problema ocurrió en la validación, en PostgreSQL o en la escritura en MongoDB.

Una métrica, en cambio, es una medición de un servicio capturada durante su ejecución, con su valor, la hora y metadatos, y sirve como indicador de disponibilidad y rendimiento [7]. A diferencia de la traza, que sigue el recorrido de una solicitud individual, las métricas entregan información estadística agregada, por ejemplo la tasa de solicitudes o el tiempo de respuesta promedio [7].

C. Componentes de OpenTelemetry

OpenTelemetry está formado por varios componentes. La API define los tipos de datos y las operaciones para generar y correlacionar los datos de seguimiento, trazas, métricas y registros, y el SDK es su implementación en cada lenguaje: se encarga de la configuración, el procesamiento y la exportación de los datos [8]. Un exportador envía la telemetría a un consumidor, el Collector o un backend de análisis [6], [7]. El Collector es una implementación que ofrece OpenTelemetry independiente del proveedor que recibe, procesa y exporta datos de telemetría, y su uso en producción es una buena práctica [8]. Así, los datos viajan

desde la aplicación instrumentada a la API y al SDK, de ahí a un exportador y, opcionalmente, al Collector antes de llegar al backend.

D. Propagación de contexto y correlación entre señales

El contexto es un objeto que contiene la información necesaria para que el servicio emisor y el receptor puedan correlacionar una señal con otra: incluye el identificador de la traza y el del span, permitiendo rastrear el flujo completo de una solicitud [3]. Cuando un servicio llama a otro, el receptor usa esos valores para crear un span que pertenece a la misma traza, con el span del emisor como padre [3]. La propagación es el mecanismo que transfiere la información importantes que tiene el objeto del contexto entre servicios y procesos, y normalmente la realizan las bibliotecas de instrumentación de forma transparente [3]. Por defecto, OTel usa los encabezados definidos por el estándar W3C Trace Context [9], [10]. Gracias al contexto, el SDK puede insertar el identificador de la traza y del span en los registros, lo que permite ver cada registro dentro de la traza a la que pertenece [3]. En las métricas, el contexto permite agregar las mediciones según el recorrido de las llamadas, por ejemplo el tiempo de respuesta de un servicio según quién lo llamó [3].

E. Estrategias de instrumentación

La instrumentación consiste en hacer que el código emita telemetría. OpenTelemetry ofrece dos formas para hacerlo: la instrumentación basada en código y la instrumentación sin código, también llamada automática [4].

La instrumentación sin código agrega las capacidades de la API y el SDK a la aplicación, normalmente mediante un agente o una instalación similar. El tipo de mecanismo depende del lenguaje y puede ir desde la manipulación de bytecode y el monkey patching hasta el uso de eBPF [11]. Por lo general, esta vía instrumenta las bibliotecas que usa la aplicación, como las solicitudes y respuestas, las llamadas a bases de datos y a colas de mensajes, más no el propio código del sistema [11]. Se configura mediante variables de entorno u otros mecanismos depende del lenguaje, y para comenzar basta con definir un nombre de servicio [11].

La instrumentación basada en código, en cambio, utiliza directamente la API y el SDK dentro de la aplicación. Permite obtener información más detallada y completa del sistema, y la documentación oficial la presenta como un complemento esencial de la telemetría que producen las soluciones sin código [4]. Con ella, el equipo decide qué operaciones de negocio merecen un span propio y qué atributos registrar.

Se puede utilizar una estrategia híbrida que combina la instrumentación automática, que cubre las bibliotecas y el framework, con instrumentación manual en los puntos críticos de la lógica propia. En la sección III se contrastan las tres opciones según esfuerzo de adopción, nivel de detalle, mantenimiento e impacto en el rendimiento.

F. Observabilidad y costo de la instrumentación

La observabilidad se entiende como la capacidad de entender un sistema desde afuera y resolver problemas imprevistos [1]. Los registros por sí solos suelen carecer del contexto necesario para reconstruir la ejecución, y resultan más útiles cuando se relacionan con una traza y un span [1]. OpenTelemetry aborda esa relación mediante la propagación de contexto, que permite vincular las distintas señales con la solicitud que las originó [3], y lo hace con un diseño independiente del proveedor [2]. En sistemas distribuidos modernos, el rastreo distribuido es una pieza importante para lograr observabilidad, pero su adopción implica un costo: puede afectar el rendimiento y la latencia de las aplicaciones monitoreadas [12].

III. ANÁLISIS COMPARATIVO

Para comparar la instrumentación automática, manual e híbrida se usan cuatro criterios: (1) nivel de detalle, o sea, qué parte del sistema se puede ver; (2) complejidad, costo de adopción y mantenimiento; (3) rendimiento; y (4) seguridad y privacidad de los datos de telemetría. No se toma en cuenta el costo de licencias, porque OTel es de código abierto [2].

A. Nivel de detalle

La instrumentación automática entrega telemetría de las bibliotecas que usa la aplicación, como las solicitudes, las llamadas a bases de datos y a colas de mensajes, pero no del código propio del sistema [11]. En Python, el agente usa principalmente *monkey patching* y cubre bibliotecas populares como Django [13]. La manual da información más detallada y completa [4], porque el equipo decide qué spans y atributos crear. La híbrida usa la automática para la cobertura general y agrega spans manuales solo en las operaciones de negocio que lo necesitan.

B. Complejidad, costo de adopción y mantenimiento

En Python, la automática se logra instalando el distro y el exportador, ejecutando el comando de *bootstrap* y arrancando la aplicación con *opentelemetry-instrument* [13]. Se configura con variables de entorno, y para empezar solo se necesita un nombre de servicio [11], sin cambiar el código [4]. La manual obliga al equipo a escribir y mantener código con la API y el SDK, y el SDK es el que se encarga de la configuración, el procesamiento y la exportación [4],

[8]. Sobre el mantenimiento, como el agente modifica funciones de las bibliotecas mientras la aplicación se ejecuta [13], hay que mantener actualizadas las bibliotecas de instrumentación junto con las demás dependencias. En la manual, el código de telemetría queda mezclado con la lógica de negocio y hay que revisarlo cada vez que esta cambia. La híbrida junta ambos esfuerzos, pero la parte manual se reduce a pocos puntos.

C. Rendimiento

Un estudio reciente midió que el rastreo con instrumentación automática bajó el rendimiento de microservicios entre 19.55 % y 80.18 %, y que los *frameworks* de alto nivel, como Flask, se afectaron más que los de bajo nivel, como Go [12]. En ese mismo estudio, la mayor parte del costo vino de exportar los datos, y también influyeron la frecuencia y el nivel de detalle de los puntos de traza [12]. Esto indica que cualquier estrategia que agregue más spans aumenta el costo, así que la manual (y la parte manual de la híbrida) debería limitarse a las operaciones críticas. Hay que aclarar que los autores no compararon directamente la instrumentación manual con la automática, y que sus pruebas usaron *endpoints* simples con la carga al máximo, por lo que las cifras sirven como referencia y no como predicción para un sistema concreto.

D. Seguridad y privacidad

Recolectar telemetría siempre trae el riesgo de capturar información sensible o personal, y OTel no puede saber por sí solo qué datos lo son en cada caso [14]. El equipo debe revisar qué datos emiten las bibliotecas de instrumentación que usa, y recolectar solo lo necesario para observar el sistema [14]. Además, el Collector tiene procesadores que permiten limpiar datos sensibles antes de exportarlos [14]. En la automática hay que revisar lo que emiten las bibliotecas; en la manual, los atributos que el propio equipo decide registrar.

TABLA I

Comparación de las estrategias de instrumentación según criterios técnicos

Criterio	Automática	Manual	Híbrida
Nivel de detalle	Bibliotecas y <i>framework</i> ; no el código propio [11]	Detalle completo, definido por el equipo [4]	Cobertura automática más detalle en puntos críticos
Complejidad, costo y mantenimiento	Baja: instalación y configuración [13]; hay que actualizar las bibliotecas de instrumentación	Alta: código con API y SDK [4], [8]; más código propio por mantener	Media: poco código propio
Rendimiento	Costo medible [12]	Crece con el número de spans [12]	Se controla limitando los spans manuales
Seguridad	Revisar lo que emiten las bibliotecas [14]	El equipo decide qué atributos registrar [14]	Ambas revisiones

Valoración propia a partir de [4], [8], [11]–[14].

IV. APLICACIÓN A ALIMENTACR

A. Contexto del sistema

AlimentaCR es una aplicación web para gestionar donaciones de alimentos entre organizaciones donantes y beneficiarias. Su backend utiliza Django REST Framework y se organiza en capas de presentación, servicios de negocio y repositorios. PostgreSQL almacena los datos transaccionales, mientras que MongoDB se utiliza para la bitácora de eventos. Esta estructura permite separar responsabilidades, pero diagnosticar un fallo requiere identificar en qué punto del recorrido de una solicitud ocurrió y qué operaciones lo precedieron.

La publicación de una donación constituye un caso adecuado para analizar la propuesta. Esta operación requiere recibir los datos, validar reglas de negocio y guardar la donación. Entre las validaciones se encuentran la cantidad positiva, la categoría activa, la fecha límite futura y la autorización de la organización para donar. Observar únicamente la respuesta HTTP no permite distinguir cuánto tiempo consumió cada etapa ni localizar con suficiente detalle un fallo inesperado.

B. Decisión de instrumentación

A partir del análisis comparativo, se propone adoptar una estrategia híbrida. La instrumentación automática ofrecería una cobertura inicial de Django y de los clientes de bases de datos compatibles con las bibliotecas utilizadas. La instrumentación manual complementaría esa cobertura mediante spans en operaciones relevantes del negocio, como publicar una donación o aceptar una solicitud [4], [13].

Esta decisión busca equilibrar el detalle obtenido y el esfuerzo de mantenimiento. La instrumentación exclusivamente automática facilitaría la adopción, pero no describiría por sí sola las etapas internas de las reglas de negocio. Una estrategia completamente manual exigiría mantener más código de telemetría. La combinación permitiría aprovechar la cobertura de las bibliotecas y concentrar el trabajo propio en los puntos que aportan información útil para el diagnóstico.

C. Instrumentación del flujo de publicación

En el flujo propuesto, la solicitud HTTP de publicación iniciaría un span de servidor. Dentro de ese contexto se crearía un span manual para la operación de negocio, denominado `publicar_donacion`. Cuando resulte necesario distinguir sus tiempos o errores, podrían incorporarse spans hijos para la validación y la persistencia. Las operaciones de bases de datos quedarían registradas mediante la instrumentación disponible para sus clientes, evitando duplicar spans que ya produzcan las bibliotecas.

Si una regla de negocio rechaza la solicitud, la telemetría permitiría distinguir ese resultado esperado de una falla técnica, como la indisponibilidad de PostgreSQL. Los atributos describirían la operación y el tipo de resultado mediante valores controlados. No se registrarían contraseñas, tokens, cuerpos completos de solicitudes ni datos personales innecesarios [14].

Aunque AlimentaCR concentra su lógica en una aplicación Django, las trazas permitirían observar sus operaciones internas y llamadas a dependencias. Si posteriormente se incorporan servicios externos, la propagación de contexto permitiría mantener la relación entre los spans de los distintos servicios [3]. La propuesta no requiere dividir previamente el sistema en microservicios.

D. Correlación de logs, métricas y trazas

La integración de logging se configuraría para incluir `trace_id` y `span_id` cuando exista un span activo. Así, un registro de error podría relacionarse con la solicitud y con la operación donde se produjo [6]. Esta información

complementaría la respuesta del manejador global de errores y facilitaría el diagnóstico interno.

Las métricas permitirían observar el volumen de solicitudes, la proporción de errores y la distribución de los tiempos de respuesta. Frente a un aumento de latencia, el equipo revisaría las trazas del mismo servicio, operación y período, y consultaría sus logs asociados. Cuando el SDK y el backend seleccionados lo permitan, los exemplars ofrecerían un vínculo desde determinadas mediciones hacia sus trazas. Se evitaría utilizar identificadores únicos de solicitudes como etiquetas de métricas, para no multiplicar innecesariamente las series.

La bitácora de MongoDB conservaría su función de registrar eventos del negocio. La telemetría tendría una finalidad complementaria: explicar cómo se ejecutó una operación y ayudar a detectar fallos o demoras. Por ello, OpenTelemetry no sustituiría la bitácora.

E. Adopción gradual y límites

Se propone comenzar con un piloto en la publicación de donaciones. La telemetría se exportaría hacia un Collector y un backend de consulta, manteniendo la configuración de exportación separada de las reglas de negocio [8]. La elección del backend consideraría las señales y funciones de correlación necesarias, sin requerir una comparación exhaustiva de proveedores comerciales.

La adopción implica configurar dependencias, mantener la instrumentación y administrar el procesamiento, almacenamiento y retención de los datos. Para limitar estos costos se definirían políticas de muestreo de trazas, exportación por lotes y recolección de atributos necesarios. También se revisaría la eliminación de información sensible antes de exportarla [14].

Antes de ampliar la cobertura, se compararía el comportamiento del flujo con y sin instrumentación bajo una carga equivalente. Se evaluarían latencia, consumo de recursos y utilidad de la información para localizar fallos. Este artículo presenta una propuesta de adopción; no se ha implementado ni medido OpenTelemetry en AlimentaCR, por lo que los beneficios esperados requieren validación.

V. CONCLUSIONES

El análisis permite concluir que el aporte de OpenTelemetry consiste en facilitar una instrumentación común y la correlación de señales que describen aspectos diferentes de la ejecución. Las métricas muestran tendencias agregadas, las trazas permiten examinar operaciones particulares y los logs aportan información sobre eventos específicos. Su

utilidad conjunta depende de conservar el contexto y configurar las integraciones necesarias.

Para AlimentaCR se recomienda una estrategia híbrida: instrumentación automática para el framework y las bibliotecas compatibles, complementada con spans manuales en operaciones relevantes del negocio. Esta alternativa ofrece un equilibrio razonable entre cobertura diagnóstica y mantenimiento, siempre que se limite la recolección a información útil y se controle la exposición de datos sensibles.

La principal limitación del estudio es la ausencia de una implementación y de mediciones propias en AlimentaCR. Los resultados de rendimiento de la literatura corresponden a otros entornos y no permiten predecir directamente la sobrecarga del proyecto. Además, la cobertura y la correlación disponibles dependen de las bibliotecas, la configuración y el backend utilizado.

Como trabajo futuro, se propone instrumentar el flujo de publicación de donaciones, simular fallos y comparar su rendimiento con una ejecución sin instrumentación. Esta prueba permitiría evaluar la utilidad diagnóstica y ajustar el muestreo, los atributos y la retención antes de extender la propuesta a otras operaciones.

REFERENCIAS

- [1] OpenTelemetry, «Observability primer», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/observability-primer/> [Accedido: 05-oct-2026].
- [2] OpenTelemetry, «Documentation», 2025. [En línea]. Disponible en: <https://opentelemetry.io/docs/> [Accedido: 05-oct-2026].
- [3] OpenTelemetry, «Context propagation», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/context-propagation/> [Accedido: 05-oct-2026].
- [4] OpenTelemetry, «Instrumentation», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/instrumentation/> [Accedido: 05-oct-2026].
- [5] OpenTelemetry, «Traces», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/signals/traces/> [Accedido: 05-oct-2026].

- [6] OpenTelemetry, «Logs», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/signals/logs/> [Accedido: 05-oct-2026].
- [7] OpenTelemetry, «Metrics», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/signals/metrics/> [Accedido: 05-oct-2026].
- [8] OpenTelemetry, «Components», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/components/> [Accedido: 06-oct-2026].
- [9] OpenTelemetry, «Propagators API», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/specs/otel/context/api-propagators/> [Accedido: 06-oct-2026].
- [10] W3C, «Trace Context», Recomendación del W3C, 2020. [En línea]. Disponible en: <https://www.w3.org/TR/trace-context-1/> [Accedido: 06-oct-2026].
- [11] OpenTelemetry, «Zero-code», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/concepts/instrumentation/zero-code/> [Accedido: 06-oct-2026].
- [12] A. Nõu, S. Talluri, A. Iosup y D. Bonetta, «Investigating performance overhead of distributed tracing in microservices and serverless systems», en *Companion of the 16th ACM/SPEC Int. Conf. on Performance Engineering (ICPE Companion '25)*, Toronto, ON, Canadá, 2025. doi: 10.1145/3680256.3721316.
- [13] OpenTelemetry, «Python zero-code instrumentation», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/zero-code/python/> [Accedido: 06-oct-2026].
- [14] OpenTelemetry, «Handling sensitive data», 2026. [En línea]. Disponible en: <https://opentelemetry.io/docs/security/handling-sensitive-data/> [Accedido: 06-oct-2026].